

Семинар по C++ №13

Виртуальные функции

Виртуальная функция

- Тип, хотя бы с одной виртуальной функцией, является полиморфным
- Определение типа объекта происходит в рантайме
- `override/final` - подсказки для компилятора, всегда нужно писать
- Не забудьте про виртуальный деструктор
- Приватность не влияет на выбор функций в рантайме
- Виртуальными не могут быть статические функции и конструкторы
- Выбор значения аргумента по умолчанию происходит во время компиляции

dynamic cast

- Только для полиморфных типов (есть хотя бы одна виртуальная функция)
- Каст вниз по иерархии наследования
- Для указателей и ссылок
- Если ошиблись, то либо `nullptr` (для указателей) или исключение `std::bad_cast` (для ссылок)
- Делает все проверки в рантайме

RTTI

- Хранится служебная информация про тип

Виртуальные функции

```
class Base {
public:
    void foo() {
        std::cout << "Base::foo(): " << base_field << "\n";
    }
    void foo_not_override() {
        std::cout << "Base::foo_not_override()\n";
    }
    int base_field;
};

class Derived : public Base {
public:
    void foo() {
        std::cout << "Derived::foo(): " << base_field << "\n";
    }
    int derived_field;
};

int main() {
    Base b1;
    Derived d1;

    Base* imposter = reinterpret_cast<Base*>( &d1);

    b1.base_field = 10;
    d1.base_field = 20;

    b1.foo();
    d1.foo();
    imposter->foo();
}
```

```
Base::foo(): 10
Derived::foo(): 20
Base::foo(): 20
```

```
class Base {
public:
    virtual void foo() {
        std::cout << "Base::foo(): " << base_field << "\n";
    }
    virtual void foo_not_override() {
        std::cout << "Base::foo_not_override()\n";
    }
    int base_field;
};

class Derived : public Base {
public:
    void foo() override {
        std::cout << "Derived::foo(): " << base_field << "\n";
    }
    int derived_field;
};

int main() {
    Base b1;
    Derived d1;

    Base* imposter = reinterpret_cast<Base*>( &d1);

    b1.base_field = 10;
    d1.base_field = 20;

    b1.foo();
    d1.foo();
    imposter->foo();
}
```

```
Base::foo(): 10
Derived::foo(): 20
Derived::foo(): 20
```

Посмотрим в gdb

```
25 // anyway Base b1;
(gdb) ya@Kostya:~/seminars_C++/sem_11$ vim main.cpp
26 // Derived d1;
(gdb) ya@Kostya:~/seminars_C++/sem_11$ g++ -g main.cpp
(gdb) ya@Kostya:~/seminars_C++/sem_11$ ./a.out
28 Base* imposter = reinterpret_cast<Base*>(&d1);
(gdb)
30 set::foo( b1.base_field = 10;
(gdb) set print asm-demangle on
(gdb) set print demangle on
(gdb) p b1
$1 = {_vptr.Base = 0x555555557d40 <vtable for Base+16>, base_field = 1431655536}
(gdb) p d1
$2 = {<Base> = {_vptr.Base = 0x555555557d20 <vtable for Derived+16>, base_field = 1431654688}, derived_field = 21845}
```

```
(gdb) x/15xg 0x555555557d10
0x555555557d10 <vtable for Derived>: 0x0000000000000000 0x0000555555557d50
0x555555557d20 <vtable for Derived+16>: 0x00005555555553d8 0x00005555555553b2
0x555555557d30 <vtable for Base>: 0x0000000000000000 0x0000555555557d68
0x555555557d40 <vtable for Base+16>: 0x0000555555555368 0x00005555555553b2
0x555555557d50 <typeinfo for Derived>: 0x00007ffff7f91c98 0x0000555555556068
0x555555557d60 <typeinfo for Derived+16>: 0x0000555555557d68 0x00007ffff7f91008
0x555555557d70 <typeinfo for Base+8>: 0x0000555555556071 0x0000000000000001
0x555555557d80: 0x0000000000000001
```

Пояснение

- Derived:
 - top offset
 - ptr to Derived typeinfo
 - ptr to Derived::foo
 - ptr to Base::foo_not_override
- Base:
 - top offset
 - ptr to Base typeinfo
 - ptr to Base::foo
 - ptr to Base::foo_not_override

```
(gdb) x/15xg 0x55555557d10
0x55555557d10 <vtable for Derived>: 0x0000000000000000 0x000055555557d50
0x55555557d20 <vtable for Derived+16>: 0x0000555555553d8 0x0000555555553b2
0x55555557d30 <vtable for Base>: 0x0000000000000000 0x000055555557d68
0x55555557d40 <vtable for Base+16>: 0x000055555555368 0x0000555555553b2
0x55555557d50 <typeinfo for Derived>: 0x00007ffff7f91c98 0x000055555556068
0x55555557d60 <typeinfo for Derived+16>: 0x000055555557d68 0x00007ffff7f91008
0x55555557d70 <typeinfo for Base+8>: 0x000055555556071 0x0000000000000001
0x55555557d80: 0x0000000000000001
```

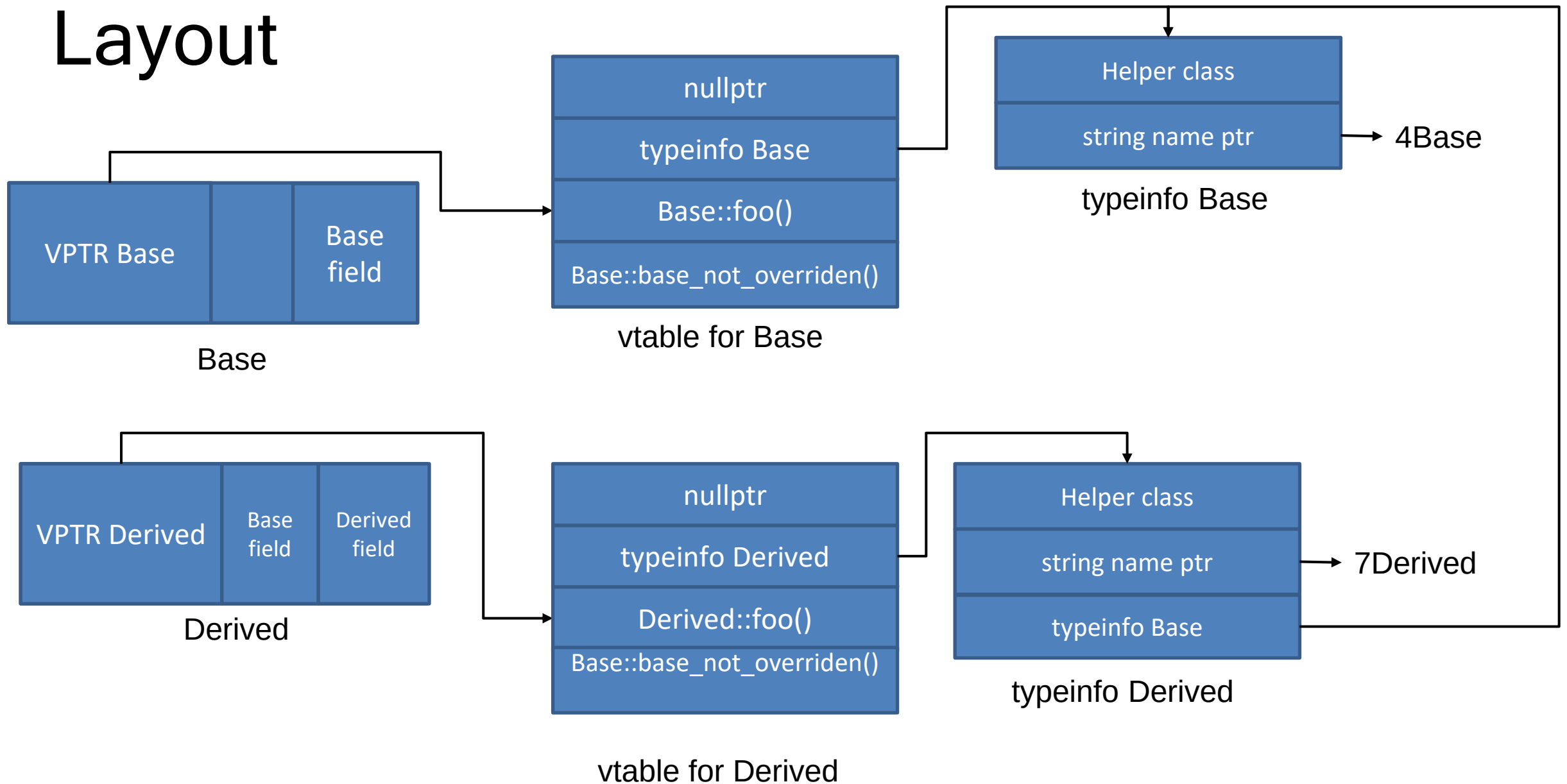
- set print asm-demangle on
- set print demangle on
- info symbol 0x...

Example

```
class Base {  
public:  
    virtual void foo();  
    void base_foo_not_virtual();  
    virtual void base_not_overriden();  
    int base_field;  
};
```

```
class Derived : public Base {  
public:  
    void foo() override;  
    void derived_foo_not_virtual();  
    int derived_field;  
};
```


Layout



Вызов методов напрямую

- Мы полезем в VTable только в 3 случае, так как компилятор всё знает про объекты уже во время компиляции.
- (Проверить это можно, например, посмотрев ассемблерный код в gdb)

```
Base b1;  
Derived d1;  
b1.foo();  
d1.foo();  
imposter->foo();
```

```
0x55555555252 <main()+73> lea    -0x30(%rbp),%rax  
0x55555555256 <main()+77> mov    %rax,%rdi  
0x55555555259 <main()+80> callq 0x55555555368 <Base::foo()>_11  
0x5555555525e <main()+85> lea    -0x20(%rbp),%rax  
0x55555555262 <main()+89> mov    %rax,%rdi  
0x55555555265 <main()+92> callq 0x555555553d8 <Derived::foo()>  
0x5555555526a <main()+97> mov    -0x38(%rbp),%rax  
0x5555555526e <main()+101> mov    (%rax),%rax  
0x55555555271 <main()+104> mov    (%rax),%rdx  
0x55555555274 <main()+107> mov    -0x38(%rbp),%rax  
0x55555555278 <main()+111> mov    %rax,%rdi  
0x5555555527b <main()+114> callq  *%rdx
```

Дальнейшее чтение

- Если вас заинтересовало происходящее и вы хотите узнать больше про виртуальные функции, прочитайте [цикл статей](#).
- Про helper class для typeid [тут](#) и [там](#)